that the gateway can translate from a more stable, client-friendly, public API to the internal one.

Suppose you're in the process of moving from a monolith to microservices. In that case, an API gateway can be used as a *monolith strangler* and can wrap your legacy applications until they are migrated to the new architecture, keeping the process transparent to clients. In case of different client types (single-page applications, mobile applications, desktop applications, IoT devices), an API gateway gives you the option to provide a better-crafted API to each of them depending on their needs (also called the *backend-for-frontend* pattern). Sometimes a gateway can also implement the *API composition* pattern, letting you query and join data from different services before returning the result to a client (for example, using the new Spring for GraphQL project).

Calls are forwarded to downstream services from the gateway according to specified routing rules, similar to a reverse proxy. This way the client doesn't need to keep track of the different services involved in a transaction, simplifying the client's logic and reducing the number of calls it has to make.

Since the API gateway is the entry point to your system, it can also be an excellent place to handle cross-cutting concerns like security, monitoring, and resilience. *Edge servers* are applications at the edge of a system that implement aspects like API gateways and cross-cutting concerns. You can configure circuit breakers to prevent cascading failures when invoking the services downstream. You can define retries and timeouts for all the calls to internal services. You can control the ingress traffic and enforce quota policies to limit the use of your system depending on some criteria (such as the membership level of your users: basic, premium, pro). You can also implement authentication and authorization at the edge and pass tokens to downstream services (as you'll see in chapters 11 and 12).